Lecture 2 Compendium: Technological Trends & Future Architectures

AEC 813: Artificial Intelligence, IoT, and GIS Application in Agriculture

Dr. Kadeghe Fue
PhD Agroecology

Module 1: Introduction & Basics

Contents

1 Introduction: The Evolutionary Trajectory **2**
1.1 The Five Ages of Agriculture 2

2 Trend 1: Internet of Underground Things (IoUT) **2**
2.1 The Physics of Signal Attenuation in Soil 2
2.2 Python Simulation: Sub-Surface Attenuation 3

3 Trend 2: Edge AI vs. Cloud AI **3**
3.1 Latency Mathematics 4
3.2 Python Benchmark: The Cost of Latency 4

4 Trend 3: Sensor Fusion & Digital Twins **4**
4.1 Inverse Variance Weighting 5
4.2 Python: Sensor Fusion Demo 5

5 Critical Analysis: The Integration Gap **5**

6 Review Questions **6**

1 Introduction: The Evolutionary Trajectory

To conduct meaningful research in AgTech, one must understand the trajectory of the technology. We are currently witnessing a paradigm shift from **Agriculture 4.0** (Connected Systems) to **Agriculture 5.0** (Autonomous Systems).

1.1 The Five Ages of Agriculture

- **Ag 1.0 (Manual):** Labor-intensive, low productivity. Characterized by human and animal muscle power. (Pre-1920s)
- **Ag 2.0 (Mechanized):** The Green Revolution. Introduction of combustion engines, synthetic fertilizers, and hydraulics. (1920s-1990s)
- **Ag 3.0 (Precision):** Guidance Systems. Integration of GNSS (GPS) for auto-steering and Variable Rate Technology (VRT) based on static maps. (1990s-2010s)
- **Ag 4.0 (Connected):** The era of "Dashboards." IoT sensors, Cloud Computing, and Big Data allow for real-time monitoring. (2010s-Present)
- **Ag 5.0 (Autonomous):** The era of "Action." Robotics, Edge AI, and Digital Twins allow systems to act without human intervention. (Future/Emerging)

PhD Research Context

The Leapfrog Effect: In developing contexts like Tanzania, the progression is rarely linear. A smallholder farmer may bypass heavy mechanization (Ag 2.0) and jump directly to using a smartphone-controlled solar irrigation system (Ag 4.0). Your research should focus on these hybrid architectures.

2 Trend 1: Internet of Underground Things (IoUT)

Standard IoT operates in the air, where RF waves propagate relatively easily. **IoUT** involves burying sensors to monitor the root zone. This creates a hostile environment for electromagnetic communication.

2.1 The Physics of Signal Attenuation in Soil

The primary antagonist for underground wireless communication is water. Water has a high dielectric constant ($\epsilon_r \approx 80$), whereas dry soil is low ($\epsilon_r \approx 3-5$). High permittivity causes rapid signal attenuation.

The **Complex Dielectric Constant** of soil is given by:

$$\epsilon_{soil} = \epsilon' - j\epsilon'' \tag{1}$$

Where ϵ' represents energy storage (capacitance) and ϵ'' represents energy loss (attenuation).

The **Modified Friis Transmission Equation** for an underground link is:

$$P_{rx} = P_{tx} + G_{tx} + G_{rx} - (L_{fspl} + L_{soil} + L_{reflection}) \tag{2}$$

Where L_{soil} (Soil Path Loss) is defined by the attenuation factor α (dB/m):

$$L_{soil} = \alpha \cdot d \tag{3}$$

α increases drastically with:

1. **Frequency (f):** Higher frequencies (2.4GHz WiFi) attenuate much faster than Sub-GHz (433MHz LoRa).
2. **Volumetric Water Content (θ_v):** Wet soil acts like a Faraday cage.

2.2 Python Simulation: Sub-Surface Attenuation

We simulate the survivability of a signal at various depths for two common frequencies: 2.4 GHz (Zigbee/WiFi) vs 433 MHz (LoRa).

```python
import numpy as np
import matplotlib.pyplot as plt

def soil_signal_simulation():
    # Parameters
    tx_power = 10 # dBm (Standard ESP32/Arduino max)
    sensitivity_limit = -100 # dBm (Receiver floor)

    # Depths to simulate (0.01m to 1.0m)
    depths = np.linspace(0.01, 1.0, 100)

    # Attenuation Factors (alpha in dB/m)
    # These are approximations based on Peplinski's model for Loam
    alpha_24_dry = 10   # 2.4GHz Dry
    alpha_24_wet = 100  # 2.4GHz Wet (Massive loss)
    alpha_433_dry = 2   # 433MHz Dry
    alpha_433_wet = 25  # 433MHz Wet

    # Calculate Received Signal Strength Indicator (RSSI)
    # RSSI = Tx - (Alpha * Depth)
    rssi_24_dry = tx_power - (alpha_24_dry * depths)
    rssi_24_wet = tx_power - (alpha_24_wet * depths)
    rssi_433_wet = tx_power - (alpha_433_wet * depths)

    # Plotting
    plt.figure(figsize=(10, 6))
    plt.plot(depths*100, rssi_24_dry, 'g--', label='2.4GHz (Dry)')
    plt.plot(depths*100, rssi_24_wet, 'g-', label='2.4GHz (Wet)')
    plt.plot(depths*100, rssi_433_wet, 'b-', label='433MHz (Wet)')
    plt.axhline(y=sensitivity_limit, color='r', linestyle=':', label='Signal
    Death')

    plt.title('IoUT: Signal Survival vs. Burial Depth')
    plt.xlabel('Depth (cm)')
    plt.ylabel('RSSI (dBm)')
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.show()

soil_signal_simulation()
```

Listing 1: Simulating IoUT Signal Loss

Interpretation: You will observe that 2.4GHz signals die within 15-20cm in wet soil, making standard WiFi sensors useless for root-zone monitoring during the rainy season. 433MHz signals penetrate significantly deeper.

3 Trend 2: Edge AI vs. Cloud AI

The traditional IoT model is "Dumb Sensor → Smart Cloud." The emerging trend is "Smart Sensor → Action." This is driven by the latency requirements of robotics.

3.1 Latency Mathematics

Why do we need Edge AI? Because the cloud is too slow for real-time actuation (e.g., a tractor spot-spraying weeds at speed).

The Total Latency (T_{total}) equation is:

$$T_{total} = T_{capture} + T_{uplink} + T_{cloud_proc} + T_{downlink} + T_{actuate} \tag{4}$$

- **Cloud Scenario:** T_{uplink} (4G) can vary from 100ms to 2000ms.
- **Edge Scenario:** T_{uplink} and $T_{downlink}$ are eliminated. $T_{total} \approx T_{proc}$.

3.2 Python Benchmark: The Cost of Latency

We simulate a tractor moving at 2 m/s. We calculate the "Miss Distance" (how far the tractor moves between seeing a weed and the sprayer activating).

```python
import numpy as np

def latency_benchmark():
    n_samples = 1000
    tractor_speed_mps = 2.0 # 2 meters/second (~7.2 km/h)

    # --- MODEL 1: EDGE AI (On-Device) ---
    # Processing time on Jetson Nano / Raspberry Pi
    # Mean: 50ms, Std: 5ms
    edge_times = np.random.normal(0.050, 0.005, n_samples)

    # --- MODEL 2: CLOUD AI (4G/LTE) ---
    # Network is unstable in rural areas (LogNormal distribution)
    # Mode at 500ms, long tail for bad packets
    network_latency = np.random.lognormal(mean=np.log(0.5), sigma=0.5, size=
    n_samples)
    cloud_proc = 0.100 # Cloud GPU is fast (100ms)
    cloud_times = network_latency + cloud_proc

    # Calculate Miss Distance
    edge_miss = edge_times * tractor_speed_mps
    cloud_miss = cloud_times * tractor_speed_mps

    print(f"--- BENCHMARK RESULTS (Speed: {tractor_speed_mps} m/s) ---")
    print(f"Edge Miss Distance: {np.mean(edge_miss)*100:.1f} cm")
    print(f"Cloud Miss Distance: {np.mean(cloud_miss)*100:.1f} cm")

    # Success Rate (Assuming 10cm wide weed, +/- 5cm tolerance)
    edge_success = np.sum(edge_miss < 0.10) / n_samples * 100
    cloud_success = np.sum(cloud_miss < 0.10) / n_samples * 100

    print(f"Edge Spray Accuracy: {edge_success:.1f}%")
    print(f"Cloud Spray Accuracy: {cloud_success:.1f}%")

latency_benchmark()
```

Listing 2: Edge vs Cloud Latency Simulation

Conclusion: Cloud AI is mathematically inviable for high-speed precision weeding. The "Miss Distance" is often meters, not centimeters.

4 Trend 3: Sensor Fusion & Digital Twins

No single sensor provides the complete picture.

- **Satellite (Macro):** Broad coverage, low resolution, blocked by clouds.
- **Drone (Meso):** High resolution, on-demand, limited battery.
- **IoT (Micro):** Continuous temporal data, spatially sparse (point data).

Sensor Fusion is the mathematical combination of these sources to reduce uncertainty (Variance).

4.1 Inverse Variance Weighting

A simple yet powerful form of fusion. We weight sensors based on their certainty (Inverse Variance). A precise sensor gets a higher weight.

$$\hat{x}_{fused} = \frac{\sum w_i x_i}{\sum w_i} \quad \text{where} \quad w_i = \frac{1}{\sigma_i^2} \tag{5}$$

4.2 Python: Sensor Fusion Demo

Combining a noisy Satellite reading with a precise Drone reading.

```python
def sensor_fusion_demo():
    true_Nitrogen = 120 # kg/ha

    # 1. Satellite Measurement (High Variance)
    # e.g., Atmospheric interference, mixed pixels
    sat_sigma = 20
    sat_reading = np.random.normal(true_Nitrogen, sat_sigma)

    # 2. Drone Measurement (Low Variance)
    # e.g., High res multispectral
    drone_sigma = 5
    drone_reading = np.random.normal(true_Nitrogen, drone_sigma)

    # Calculate Weights
    w_sat = 1 / (sat_sigma**2)
    w_drone = 1 / (drone_sigma**2)

    # Fused Estimate
    fused_est = (w_sat * sat_reading + w_drone * drone_reading) / (w_sat +
    w_drone)

    # Variance of the fused estimate
    fused_variance = 1 / (w_sat + w_drone)
    fused_sigma = np.sqrt(fused_variance)

    print(f"Satellite: {sat_reading:.2f} (Sigma: {sat_sigma})")
    print(f"Drone:     {drone_reading:.2f} (Sigma: {drone_sigma})")
    print(f"Fused:     {fused_est:.2f} (Sigma: {fused_sigma:.2f})")

    if fused_sigma < drone_sigma:
        print("SUCCESS: Fused uncertainty is lower than any single sensor.")

sensor_fusion_demo()
```

Listing 3: Inverse Variance Weighting Fusion

5 Critical Analysis: The Integration Gap

As PhD students, you must critique current trends, not just adopt them.

1. **Interoperability Hell:** A John Deere tractor generates a shapefile. A DJI drone generates a TIFF. A Davis weather station generates a CSV. There is no unified "Ag-OS" (Operating System). Research is needed in middleware and Ontologies (e.g., SSN - Semantic Sensor Network).

2. **Data Sovereignty:** When a smallholder farmer sends data to a cloud Digital Twin, who owns the resulting insight? The multinational corporation or the farmer?

3. **Technological Debt:** In Tanzania, deploying a complex Digital Twin requires maintenance. If the PhD student graduates and leaves, the system often dies. Architects must design for *Simplicity* and *Repairability*.

6 Review Questions

1. **IoUT:** Explain why a 2.4GHz Zigbee sensor buried 30cm deep works in August (Dry season) but fails in December (Wet season) in Morogoro. Use the concept of Dielectric Constant.

2. **Edge Computing:** Calculate the maximum allowable latency for a weeding robot traveling at 5 km/h if the weeding tool has a longitudinal tolerance of $\pm$2cm.

3. **Sensor Fusion:** Why does fusing a Satellite image (10m resolution) with a Drone image (5cm resolution) improve the accuracy of yield prediction models?

References

1. Vuran, M. C., et al. (2018). *Internet of Underground Things in Agriculture: Architecture and Technology*. IEEE Communications Surveys & Tutorials.

2. Mahmud, M. S., et al. (2020). *A Review of Computer Vision and Edge Computing in Precision Agriculture*.

3. **Open Source Dataset:** SoilSCAPE (Soil moisture Sensing Controller and oPtimal Estimator) - Available at `http://soilscape.github.io/`.